Aula 5 — KMP: Kotlin Multiplatform + BFF

"Uma base de código, três plataformas"

Prof. Jackson Smith Moisés Matias
PUC Minas IEC

Recap da Aula 4

- PWA: Web App Manifest + Service Worker lifecycle
- Cache strategies: CacheFirst, NetworkFirst, StaleWhileRevalidate
- Workbox: abstração declarativa de SW
- Background Sync + IndexedDB
- **Atividade 4** (PWA + TMDB) — prazo no Canvas; dúvidas, fale comigo

O problema: 3 meses, time pequeno, 3 plataformas

> *"Recebemos aporte. Precisamos lançar o app pra* **Android, iOS e Web em 3 meses** *— mas o orçamento só dá pra contratar um time pequeno. A diretoria quer saber: nativo (3 times) ou multiplataforma (1 time, 1 base de código)? E se for multiplataforma — qual?"*

Hoje respondemos com **KMP**: 1 base de lógica compartilhada, 3 alvos de plataforma. A pergunta que interessa não é "como escrever Kotlin" — é **onde essa promessa aperta na prática**.

Agenda da aula

Bloco 1 (90min) — KMP foundations: código compartilhado entre Android, iOS e Web
→ KotlinProject · setup toolchain · arquitetura shared · Ktor + TMDB · `expect` / `actual`

Intervalo (30min)

Bloco 2 (60min) — KMP na prática + Trabalho Final
→ **Atividade 5 ao vivo** (exercício KMP) · **Projeto Final (30 pts)**: 5 temas · 3 frameworks · rubrica · logística
→ **BFF/GraphQL** (conceito)

🎯 A diretoria deu 1 semana pra provar a ideia

"Android, iOS e Web em 3 meses, time pequeno."
Antes de apostar, a diretoria quer ver **o mesmo código rodando no celular**.

Vocês são o time. Dia 1 de qualquer time: **ambiente pronto**.

Comecem AGORA, enquanto eu explico a teoria

1. **Android Studio** — `developer.android.com/studio`
2. Deixar o **SDK** baixar
3. Criar um **emulador** (ou celular + cabo USB)

> Rede ruim? Comecem já — o 1º sync baixa vários GB.

Objetivos de aprendizagem

- Compartilhar lógica de negócio entre Android, iOS e Web com Kotlin
- Integrar HTTP com Ktor + kotlinx.serialization + TMDB API
- Entender `expect` / `actual` para código platform-specific
- Entender padrão Backend-for-Frontend — um servidor, múltiplos clientes
- Comparar REST (KMP/PWA) vs GraphQL — conceito e quando usar

KotlinProject — como foi criado

Projeto gerado pelo **KMP wizard oficial** da JetBrains:

> **kmp.jetbrains.com**
> Config usada: Android ✓ · iOS ✓ · iOS UI = **Compose** · Include Tests ✓

Essa URL reproduz exatamente o starter. Qualquer aluno pode gerar o mesmo projeto do zero no wizard — é o ponto de partida oficial pra projetos KMP novos.

O que vamos rodar hoje — KotlinProject

Três apps, **um módulo compartilhado** (`shared` — Compose Multiplatform):

App	Módulo	Plataforma
`androidApp`	`:androidApp`	Android
`iosApp`	`iosApp/`	iOS (Xcode project)
`webApp`	`:webApp`	Web (JS / Wasm)

Targets declarados em `shared/build.gradle.kts`:
`androidTarget`, `iosArm64`, `iosSimulatorArm64`, `js`, `wasmJs`

A prova de conceito (~30 min)

Abrir o **KotlinProject** e rodar o `androidApp`.
É também o **passo 1 da Atividade 5**.

	Etapa
1	**Ambiente pronto** — Studio + emulador
2	**Versões certas** das ferramentas
3	**Plugin KMP** no Studio
4	▶ **Rodar no Android**

> **Opcional:** mesmo código no **iOS** (só Mac) e na **Web**.

Toolchain — por que a versão importa

Peças acopladas: versão errada = ***Sync failed***. Estas já foram testadas juntas:

Ferramenta	Versão	
Android Studio	Quail 2026.1.1 Patch 2	você instala
Gradle · AGP · Kotlin · Compose	9.1.0 · 9.0.1 · 2.4.0 · 1.11.1	já vêm no projeto
JDK	17	compila o Gradle
Xcode	26.1.1	só iOS (opcional)

> Studio **mais antigo** que o Quail 2026.1.1 → o projeto **não carrega**.

Gotchas do setup (guardem, vão aparecer)

- **Apple Silicon** usa `iosSimulatorArm64` — o `iosX64` não está declarado, então **simulador Intel não builda**
- **Config cache instável** → rode com `./gradlew --no-configuration-cache <task>`
- **Nunca commitar** `local.properties` — o caminho do SDK é de cada máquina

Setup — pré-requisitos

- **Android Studio Quail 2026.1.1+** — versão exata importa
- **JDK 17** e um **emulador Android** (ou celular com depuração USB)
- `local.properties` criado automaticamente pelo Studio:

```
sdk.dir=/Users/<seu-user>/Library/Android/sdk
```

> Não commitar — é caminho de máquina.

- **Só pra iOS (Mac):** Xcode 26.x + command-line tools — confira com:

```
xcode-select -p
# → /Applications/Xcode.app/Contents/Developer   ✓
```

- Mac: Homebrew (`/opt/homebrew/bin/brew`) · Windows/Linux: pule o que for de Xcode/iOS

Seu celular de teste: o emulador (AVD)

1. **Device Manager** (ícone de celular) → **Create Device**
2. Hardware: **Pixel 8**
3. System image: **API 34** → *Download*
   - Apple Silicon: `arm64-v8a` · Intel/AMD: `x86_64`
4. **Finish** → ▶ (o 1º boot demora alguns minutos)

> É um **celular virtual** — a prova de conceito vai rodar nele.

Não deu o emulador? Plano B

- **Celular Android + cabo USB** — ative *Depuração USB* (toque 7× em *Número da versão*)
- **Windows:** virtualização ligada na BIOS · **WSL não roda emulador**
- **Web:** acompanhe pelo meu e rode no navegador

> **Não perca a aula tentando** — avise no chat.

Setup — plugin KMP no Android Studio

1. **Settings → Plugins → Marketplace** → **Kotlin Multiplatform** → Install
2. Reiniciar o Studio
3. O dropdown de run agora mostra `androidApp` (e `iosApp`) ✓

> Erro *"Requires plugin com.intellij.marketplace"*? Instale também **JetBrains Marketplace Licensing**.

Verificar ambiente — kdoctor (opcional, só Mac)

```
brew install kdoctor
kdoctor            # ou /opt/homebrew/bin/kdoctor
```

Esperado: OS ✓ · Java ✓ · Xcode ✓

2 warnings seguros para ignorar:

- *"Android Studio: KMM plugin not installed"* — kdoctor 1.1.0 checa id legado; nosso plugin está ok
- *"CocoaPods not configured"* — este projeto não tem `Podfile`; só necessário com libs CocoaPods

iOS Simulator não existe? kdoctor avisa *"Couldn't find available Apple Simulators"* — resolve com:

```
xcrun simctl list devices available   # checar o que já existe
xcrun simctl create "iPhone 16" \
  com.apple.CoreSimulator.SimDeviceType.iPhone-16 \
  com.apple.CoreSimulator.SimRuntime.iOS-26-1
```

Rodando os 3 targets

Android

- Dropdown → `androidApp` + emulador/device → ▶️

iOS

- Dropdown → `iosApp` + `iPhone 16` → ▶️
- Fallback (sem plugin):

```
open iosApp/iosApp.xcodeproj   # Run pelo Xcode
```

Web

```
./gradlew :webApp:jsBrowserDevelopmentRun       # JS
./gradlew :webApp:wasmJsBrowserDevelopmentRun   # Wasm
```

O que acabou de rodar: a ideia do KMP

Você abriu **um projeto** e o mesmo código apareceu no celular. Como? Ele é dividido em **duas camadas**:

shared / commonMain — escrito **uma vez**
telas, models, regras de negócio, chamadas à API

▼ o compilador gera um app por plataforma ▼

androidApp	**iosApp**	**webApp**
só "hospeda" o shared	só "hospeda" o shared	só "hospeda" o shared

> É isso que responde o chamado: **1 base de lógica, 3 alvos**. A linguagem do `shared` é **Kotlin** — um tour rápido a seguir.

Kotlin — sintaxe em 1 slide

```kotlin
val nome = "Ana"            // imutável (≈ const)
var contador = 0            // mutável
fun saudar(n: String) = println("Oi, $n")    // função

data class Filme(val titulo: String, val nota: Double)   // equals/copy/toString grátis

val apelido: String? = null          // ? = pode ser null
val tam = apelido?.length ?: 0       // safe-call (?.) + elvis (?:)
listOf(1, 2, 3).filter { it > 1 }    // lambda: it = o item
```

> `val` / `var` · `data class` = seu modelo · `?` / `?:` = null-safety · `{ it }` = lambda. É a linguagem do **"shared"** do KMP.

Mas e o que *muda* de plataforma pra plataforma?

Algumas coisas **só cada SO sabe** (versão, arquivos, câmera).
O shared **pede**; cada plataforma **entrega** — como uma tomada 🔌 e o plugue certo.

- **`expect`** = "espero que exista isto" — só o nome, no `shared`
- **`actual`** = "aqui está o de verdade" — um por plataforma

```
// shared (commonMain) — o contrato
expect fun plataforma(): String

// androidMain                              // iosMain
actual fun plataforma() = "Android"         actual fun plataforma() = "iOS"

Text("Rodando em ${plataforma()}")   // Android no emulador · iOS no iPhone
```

> Faltou o `actual`? **O projeto nem compila.**

`expect` / `actual` num caso real: o cliente HTTP

```
// commonMain — o contrato
expect class HttpClient { suspend fun get(url: String): String }

// androidMain                          // iosMain
actual class HttpClient {               actual class HttpClient {
    actual suspend fun get(...)             actual suspend fun get(...)
        = /* OkHttp */                          = /* NSURLSession */
}                                       }
```

> **Na prática você raramente escreve isso:** o **Ktor** já faz por dentro (OkHttp no Android, NSURLSession no iOS). Você só usa `HttpClient()` — é o que a Atividade 5 faz.

Código por plataforma — cada framework tem uma resposta

Framework	Mecanismo	Checado em
React Native	`Platform.OS` · `.ios.tsx` / `.android.tsx`	execução
Flutter	`Platform.isIOS` · `MethodChannel`	execução
KMP	**`expect` / `actual`**	**compilação**

> Esqueceu uma plataforma? No KMP **não compila**. No RN/Flutter explode **na mão do usuário**.

KMP — as peças do ecossistema

- **Kotlin** — a linguagem (Android, iOS, Web, JVM…)
- **Compose Multiplatform** — UI compartilhada
- **Ktor** — cliente HTTP
- **kotlinx.serialization** — JSON
- **BuildKonfig** — config compartilhada (nosso `tmdb.token`)

> Compartilha **regra de negócio**; a UI compartilhada ainda está amadurecendo (no iOS).

Quem já usa KMP em produção

- **Netflix** — lógica compartilhada nos apps do estúdio
- **McDonald's** — pagamentos no app, compartilhados entre Android e iOS
- **Forbes** — mais de 80% da lógica compartilhada
- **Philips** · **Cash App** · **9GAG** · **Quizlet**

> Fonte: *case studies* oficiais em **kotlinlang.org** (números reportados pelas próprias empresas).

O gargalo real do KMP

O **Cash App** (pagamentos P2P, EUA) usa KMP desde 2018 — e **documentou** onde pega:

- **iOS interop** — debugar Kotlin pelo Xcode é mais lento
- **Compose no iOS** — bugs de renderização que o Android não tem

> **Lição:** KMP compartilha regra de negócio — **não elimina** trabalho de plataforma.

🌐 Hands-on guiado — Kotlin (Playground)

play.kotlinlang.org (zero instalação). Escreva a **"shared logic" dos favoritos** — a mesma do `favoritesStore` do app:

```kotlin
data class Movie(val id: Int, val title: String, val rating: Double)   // 1. modelo

fun favorites(movies: List<Movie>, favIds: Set<Int>) =                 // 2. função pura
  movies.filter { it.id in favIds }

fun main() {
  val all = listOf(
    Movie(1, "Matrix", 8.7),
    Movie(2, "Inception", 8.8),
    Movie(3, "Tenet", 7.4),
  )
  val favs = favorites(all, setOf(1, 3))                               // 3. rode!
  println(favs.map { it.title })   // → [Matrix, Tenet]
}
```

> (1) `data class` = modelo · (2) função pura de filtro · (3) rode. **Essa lógica o KMP compartilha** entre Android e iOS — só o que toca plataforma (rede, sensores) vira `expect` / `actual`.

☕ Intervalo — 30min

Volta às :___

Aproveita pra:

- Esticar perna
- Pegar token TMDB se não tiver: developer.themoviedb.org → API → Bearer
- Android Studio aberto com o projeto KMP carregado

> Próximo bloco: BFF/GraphQL + Atividade 5.

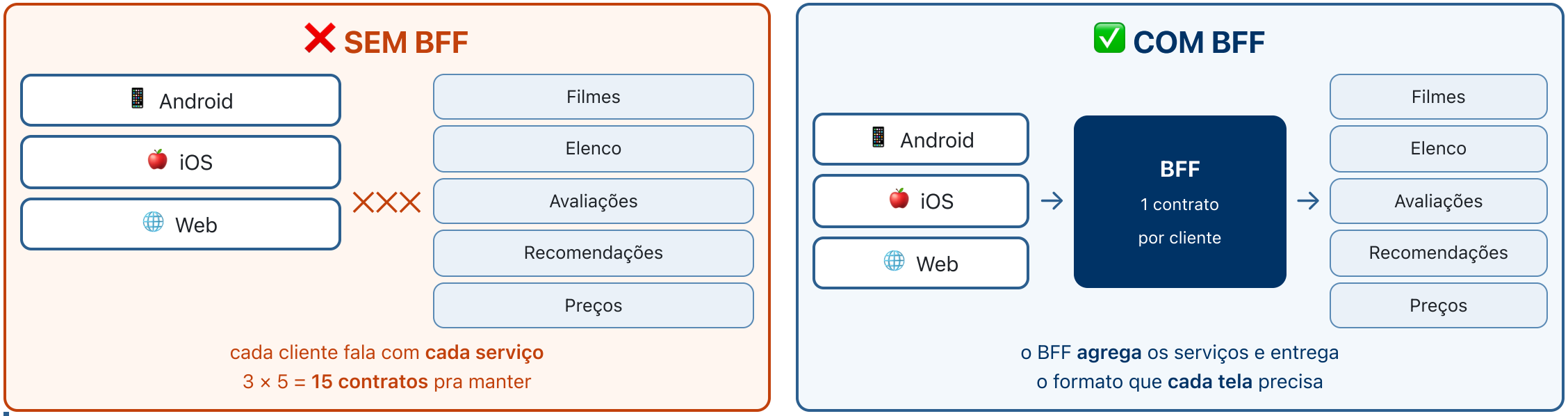
O problema que o BFF resolve

> *"Catálogo em* **5 microsserviços**, **3 clientes**, **23 endpoints REST** *sob medida — um por tela×cliente. Toda mudança de design trava esperando o backend."*

"Não dá só pra paralelizar ou criar mais 1 endpoint?" Funciona pra **uma** tela.

O que cresce sem controle é **telas × clientes × serviços**: cada combinação nova = mais 1 endpoint, 1 PR de backend, 1 contrato pra versionar.

BFF — a arquitetura



> **BFF = Backend-for-Frontend:** uma camada entre os apps e os serviços, feita **sob medida pro cliente**.

BFF — Backend-for-Frontend

O problema sem BFF

- RN, KMP, Web cada um bate em endpoints REST diferentes
- Over-fetching (campos que não usa) e under-fetching (N+1 requests)
- Múltiplos times sincronizando formatos de resposta

Com BFF + GraphQL

- Um endpoint `/graphql` serve **todos os clientes**
- Cliente pede **só os campos que precisa**
- Schema = contrato único versionado

```
# Android pergunta isso:
query {
  popularMovies {
    results { title voteAverage }
  }
}
```

```
# PWA pergunta isso:
query {
  popularMovies {
    results { title posterPath overview }
}
}
```

> **Mesma query shape, respostas diferentes.** O BFF otimiza por cliente sem mudar o backend.

GraphQL — paradigma alternativo

REST é orientado a **recursos** (`/movies/:id` , `/movies/:id/reviews`). GraphQL é orientado a **queries declarativas** — o cliente descreve o formato que quer, o servidor entrega exatamente isso.

```
query MovieDetail($id: ID!) {
  movie(id: $id) {
    id
    title
    director { id name }
    reviews(limit: 10) { id rating comment user { name } }
  }
}
```

> **Uma chamada**, exatamente os dados que o cliente precisa — nem mais, nem menos.

GraphQL — pontos fortes

- **Cliente decide** o shape do payload (sem over-fetch / under-fetch)
- **Schema strong-typed** — geração de tipos automática (TS/Kotlin) a partir do schema
- **Single endpoint** — uma URL, várias queries (REST tende a multiplicar endpoints)
- **Federation** — vários serviços expostos como 1 schema unificado (Netflix, Shopify usam pra isso)
- **Tooling forte** — Apollo Studio, GraphiQL/Sandbox, codegen

GraphQL — armadilhas (não é grátis)

> Mês 2 em produção: uma query aninhada sem DataLoader disparou **500+ chamadas** ao banco numa tela. Servidor caiu.

- ❌ **N+1** → resolve com **DataLoader**
- ❌ **Query sem limite de profundidade** → depth limit + rate limit
- ❌ **Cache mais complexo** que REST (CDN não ajuda igual)
- ❌ **Curva de aprendizado** — schema, resolvers, federation

GraphQL — quando faz sentido

Compensa quando: vários clientes (mobile/web/TV) · muitos microsserviços atrás de 1 BFF · front quer iterar sem esperar endpoint novo.

Quem usa: GitHub (API v4) · Shopify · Netflix (Federation) · Facebook (nasceu lá).

> **Time pequeno, poucos clientes?** REST + BFF simples resolve com menos operação.
> GraphQL **troca** complexidade de lugar — não elimina.

TMDB REST vs GraphQL — o mesmo problema, dois sabores

	TMDB REST (nossos exercícios)	GraphQL
Popular	`GET /movie/popular`	`query { popularMovies }`
Busca	`GET /search/movie?query=x`	`query { searchMovies(query:"x") }`
Over-fetching	Sim (resposta fixa)	Não (cliente escolhe campos)

KMP e **PWA** usam TMDB REST — simples, sem servidor local.

> **Lição:** GraphQL não substitui REST — só compensa quando a agregação multi-cliente supera o custo do schema.

O que já vem pronto pra Atividade 5: Movie + TmdbApi

```
// shared/.../data/Movie.kt — o dado que a UI vai receber
data class Movie(
    val id: Int, val title: String, val overview: String,
    val posterPath: String?, val releaseDate: String, val voteAverage: Double,
)

// shared/.../data/TmdbApi.kt — já implementado, busca de verdade via Ktor
class TmdbApi(token: String) {
    suspend fun popularMovies(page: Int = 1): MoviesResponse
}
```

> Vocês não tocam nesses dois arquivos — é o "prato pronto". A pergunta do exercício é: como transformar um `List<Movie>` numa tela?

Compose — os blocos que o exercício usa

Bloco	Pra quê	TODO
`when { … }`	escolher a tela (token / carregando / erro / lista)	1
`LazyColumn`	lista rolável	2
`Card` · `Row` · `Column`	compor o item da lista	3
`Text(maxLines = 2)`	truncar texto longo	3

```
LazyColumn {
    item { Text("Cabeçalho") }
    items(listOf("A", "B", "C")) { nome -> Text(nome) }
}
```

Compose — a ideia em 1 slide

```
@Composable                                        // toda UI é uma função assim
fun Ola(nome: String) {
    var contador by remember { mutableStateOf(0) }   // state: muda → a tela redesenha

    Column(modifier = Modifier.padding(16.dp)) {      // vertical
        Text("Olá, $nome! ($contador)")
        Button(onClick = { contador++ }) { Text("+1") }
    }
}
```

`Column` vertical · `Row` horizontal · `Box` sobrepõe · `Card` moldura · `Modifier` encadeia `.padding()` `.size()` …

🌐 Hands-on 2 — construa um MovieCard no @Preview

Diferente do Kotlin Playground (só lógica), Compose não roda no navegador — usamos o **@Preview** do Android Studio: atualiza a tela sem precisar de emulador.

```
@Preview
@Composable
fun CardPreview() {
    MaterialTheme {
        /* 👈 suas peças entram aqui */
        Text("comece aqui")
    }
}
```

> A cada peça: troca o `/* suas peças entram aqui */` → **Build** (ou ⌘+' / Ctrl+Alt+Shift+U) no painel Preview. Sem emulador, sem app rodando.

Peça 1 · `Text` (o título)

`Text` mostra uma string. O visual vem de parâmetros: `fontSize`, `fontWeight`, `color`.

```
Text("Matrix", fontWeight = FontWeight.Bold, fontSize = 22.sp)
```

> 👨‍💻 **Sua vez:** ponha o título **Matrix** no lugar do `/* suas peças entram aqui */` e dê Build. Experimente: mude o `fontSize` e adicione `color = Color.Blue` — veja a tela reagir.

Peça 2 · `Row` (a nota)

`Row { ... }` coloca composables **lado a lado** (horizontal).

```
Row {
    Text("★")
    Text(" 8.7")
}
```

> 👨‍💻 **Sua vez:** monte a linha da nota ⭐ **8.7** — dois `Text` dentro de um `Row`.

Peça 3 · `Column` (empilhar)

`Column` é o `Row` na **vertical**. Empilha os composables filhos de cima pra baixo.

```
Column {
    /* título · nota · ano */
}
```

> 👨‍💻 **Sua vez:** empilhe num `Column`: o **título** (peça 1) + a **linha da nota** (peça 2) + o **ano** → `Text("1999", color = Color.Gray)`.

Peça final · `Card` + `Modifier.padding` = MovieCard 🎉

`Card` dá a moldura com elevação; `Modifier.padding(16.dp)` dá o respiro interno. Embrulhe o `Column`:

```
Card {
    Column(modifier = Modifier.padding(16.dp)) {
        Text("Matrix", fontWeight = FontWeight.Bold, fontSize = 22.sp)
        Row { Text("★ 8.7") }
        Text("1999", color = Color.Gray)
    }
}
```

> 🎉 Vocês **construíram** um MovieCard — não copiaram. É a mesma peça que vocês montam de verdade no **TODO 3 da Atividade 5** (só troca texto fixo pelo `movie.title` / `movie.overview`).

Atividade 5 — KMP: filmes populares (10 pts)

Stack: o mesmo KotlinProject do Bloco 1 (Android/iOS/Web) + Ktor + TMDB REST API

Já pronto (`shared/src/commonMain/kotlin/.../data/`): `Movie.kt` + `TmdbApi.kt` (busca real via Ktor). **O exercício é construir a UI** em `App.kt` — 3 TODOs:

```
TODO 1  estados: token ausente / carregando / erro / lista      (bloco: when)
TODO 2  MovieList — LazyColumn com cabeçalho + itens              (bloco: LazyColumn)
TODO 3  MovieCard — poster placeholder + título + overview + nota/ano  (bloco: Card/Row/Column)
```

> Blocos de Compose de cada TODO: ver slide "Compose — os blocos que o exercício usa", 2 antes.

Setup:

```
cd exercicios/05-kmp/pratica
cp local.properties.example local.properties
# adicione seu tmdb.token= no local.properties
# Sync Gradle → Run androidApp
```

Entrega: fork → PR tocando `exercicios/05-kmp/pratica/` · Prazo: ver Canvas

Projeto Final — o que é

Um **app mobile** em **Flutter, React Native ou KMP**, com **5 features**, construído **em grupo**.

Grupo	3 a 4 pessoas
Valor	30 pts
Prazo final	**próxima aula (Aula 6)** — ver Canvas
Apresentação	**demonstração técnica** ao vivo · 10 min + 5 de perguntas
Entrega	**URL do repositório** do grupo no Canvas

Feature = o que o usuário vê e usa

#	Feature	Exemplo (app de filmes)
1	**Lista** de dados reais	filmes populares vindos da API
2	**Detalhe**	tocar abre sinopse e nota
3	**Busca**	filtra a lista pelo texto
4	**Filtro / categoria**	chips de gênero
5	**Favoritos**	★ e aba Favoritos — **sobrevive a fechar o app**

> Além disso: tratar **carregando, vazio e erro/sem internet** — nunca tela branca.

Escolham: tema pronto ou a sua API

Opção A — 5 temas prontos. Mesmas 5 features; muda só a API e o visual. Com *skeleton* de partida.

Opção B — a sua API. Qualquer tema, desde que:

- tenha **no mínimo as mesmas 5 features**
- use **dados reais e gratuitos** (sem cartão; nunca commitar chave)

> 🚫 **Não vale pegar o mesmo trabalho — alinhem entre vocês**
>
> Cada **tema + framework** é de **um grupo só**. Reserve abrindo uma **issue** no repo da disciplina: `Grupo <nomes> — <tema> + <framework>` · vale a **ordem de chegada**.

Opção A — os 5 temas prontos

Tema	Filtro natural
PokeAPI — Pokédex	`type`
Rick and Morty — personagens	`status`
TheMealDB — receitas	categoria
TheCocktailDB — drinks	categoria
Studio Ghibli — filmes	`director`

> Todos com as **mesmas 5 features**. O skeleton traz a estrutura e os `// TODO` ; **persistência, erros e UI/UX são do grupo**.

Para o app ficar bonito (o mínimo)

- **Design tokens** — cores, tipografia, espaçamento em **um lugar só**
- **Tema claro e escuro**
- **Loading com skeleton**, não só spinner
- **Imagens** com placeholder e fallback
- **Responsivo** — celular e paisagem
- **Acessibilidade** — contraste, toque ≥ 48dp
- **Ícone e splash próprios**

Prova: 3 *screenshots* no README + a demo ao vivo.

Arquitetura, ADR e testes

- **Camadas:** dados · estado/lógica · UI — **sem fetch dentro do widget**
- `ADR.md` **(1 página):** por que esse framework e esse gerenciador de estado — e **que alternativa descartaram**
- **Testes próprios** (≥ 3 casos) — vale **o que decidiram testar**
- `README.md` **:** o que faz · **como rodar** · 3 screenshots · link do ADR

Como entregar

1. **Um repositório GitHub do grupo** (todos como *collaborators*; privado → adicionem `jacksonsmith`)
2. **Branches + PRs internos** — a **Contribuição individual** olha o commit de **cada** pessoa
3. O app numa pasta, com **`README.md`** e **`ADR.md`**
4. **Abram o repo cedo** e deem push sempre — eu acompanho e aviso
5. **Colem a URL no Canvas** e **apresentem na Aula 6**

> **App que não builda = Funcionalidade zerada.** Rodem o build antes de entregar. **Sem autograder** — eu clono, buildo e avalio.

Projeto Final — rubrica (30 pts)

Critério	Pts
UI/UX — app bonito + demo	8
Arquitetura + ADR	7
Funcionalidade — 5 features, erros, build	6
Apresentação — demo técnica + perguntas	4
Contribuição individual	3
Qualidade — testes + README	2

Leituras complementares

- **JetBrains** — *Kotlin Multiplatform Docs* (kotlinlang.org/docs/multiplatform)
- **Ktor** — *ktor.io/docs/getting-started-ktor-client.html*
- **Newman, S.** — *Building Microservices* 2ed, capítulo 4 (BFF)
- **Porcello & Banks** — *Learning GraphQL* (O'Reilly)

Obrigado!

📧 jackson.96@gmail.com
🔗 linkedin.com/in/3jacksonsmith
🐙 github.com/jacksonsmith

Bom trabalho final! 🚀